

INFRASTRUCTURE AS CODE(IaC)

When you hear the term, Infrastructure as Code, what comes to mind?

Infrastructure as
Code (IaC)



Fundamentals of IaC

Infrastructure: Infrastructure refers to the combination of hardware and software components that make up the IT environment, such as servers, storage, network devices, firewall devices, routers, and more.

Provisioning: It is the process of creating infrastructure and making it available to end users.

Configuration: It is the process of configuring the provisioned IT infrastructure resources.
For example, installing and configuring a database on a server.

Automation: It is the process of automating tasks that is done manually, such as automating the installation of software packages, setting up users and permissions, or configuring network devices.

Orchestration: Orchestration is the process of coordinating multiple automation tasks. The output from one automation task can be used as input for another, allowing for a more complex and streamlined workflow.

Manual Approach



Micro-services



Infrastructure as Code(IaC) Approach

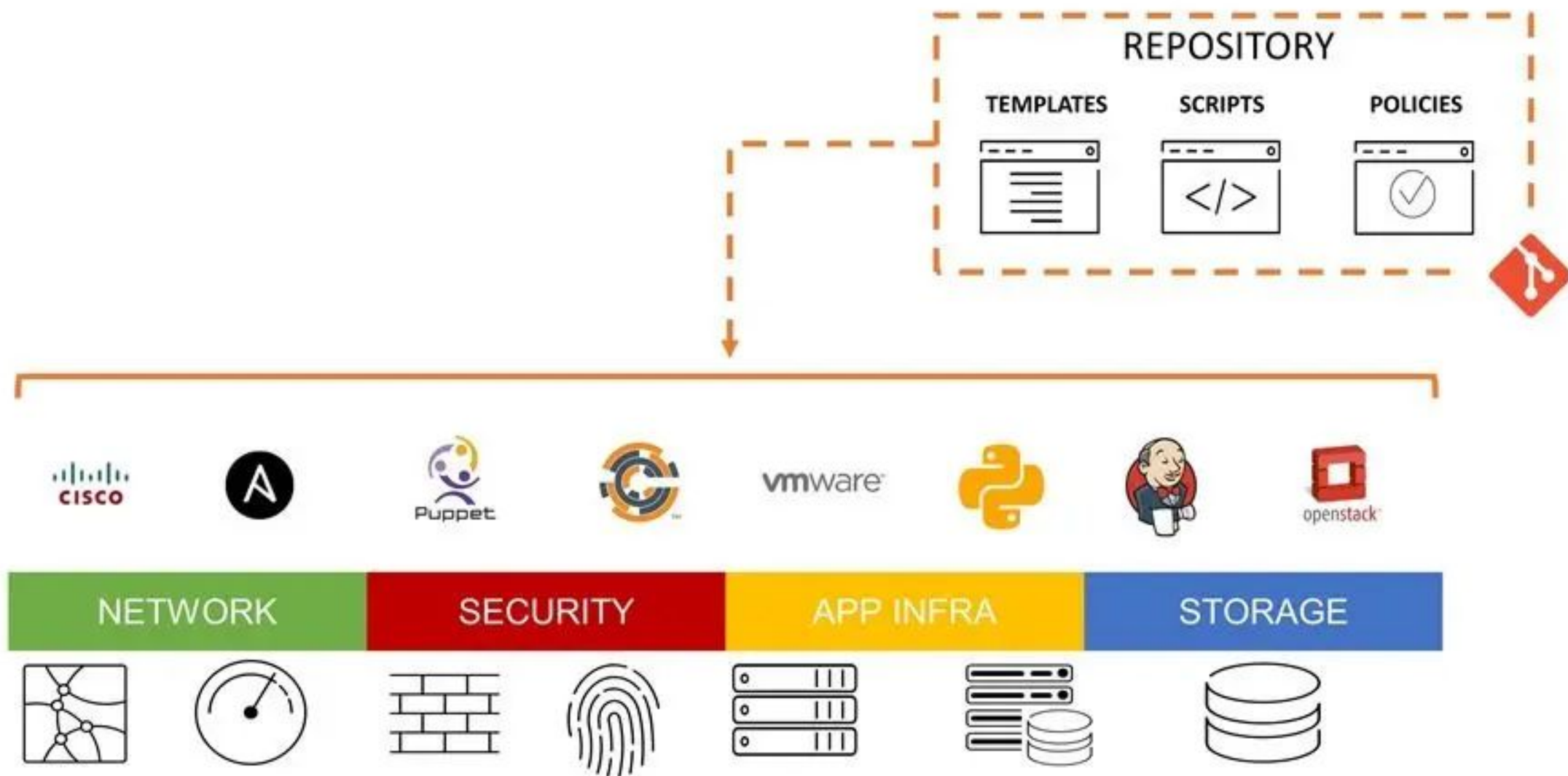


What is Infrastructure as code?

5

- Infrastructure as Code is the method of provisioning and configuring computing resources or infrastructures using code instead of manual processes.
- In simpler terms, it means treating your infrastructure — **servers, networks, databases**, and more — just like you treat your application code. Instead of manually configuring servers or setting up networks, you define your infrastructure in code, and tools take care of provisioning and maintaining it.
- Infrastructure as Code (IaC) is a core DevOps practice that enables teams to automate the provisioning and management of infrastructure using code. **This approach eliminates manual processes, improves efficiency, ensures consistency, and enhances scalability.**
- IaC allows developers and IT teams to define and deploy infrastructure in a repeatable and predictable manner, treating infrastructure configuration as software code.

INFRASTRUCTURE as CODE



Benefits of IaC in DevOps?

1. Automation

- IaC automates the process of infrastructure provisioning, making it faster, consistent, and error-free, reducing human errors and saving time.

2. Scalability and Consistency

- Through IaC, you can easily replicate your infrastructure across different environments, ensuring consistency.
- It becomes simpler to scale your infrastructure up or down to meet the demands of your application.

3. Version Control

- IaC code is typically stored in version control systems like Git.
- This enables you to track changes, collaborate with team members, and revert to previous configurations if issues arise.

4. Documentation

- Your IaC code serves as documentation for your infrastructure.
- It provides a clear and executable representation of how your infrastructure is set up, making it easier for teams to understand and manage.

5. Fast Recovery and Redundancy

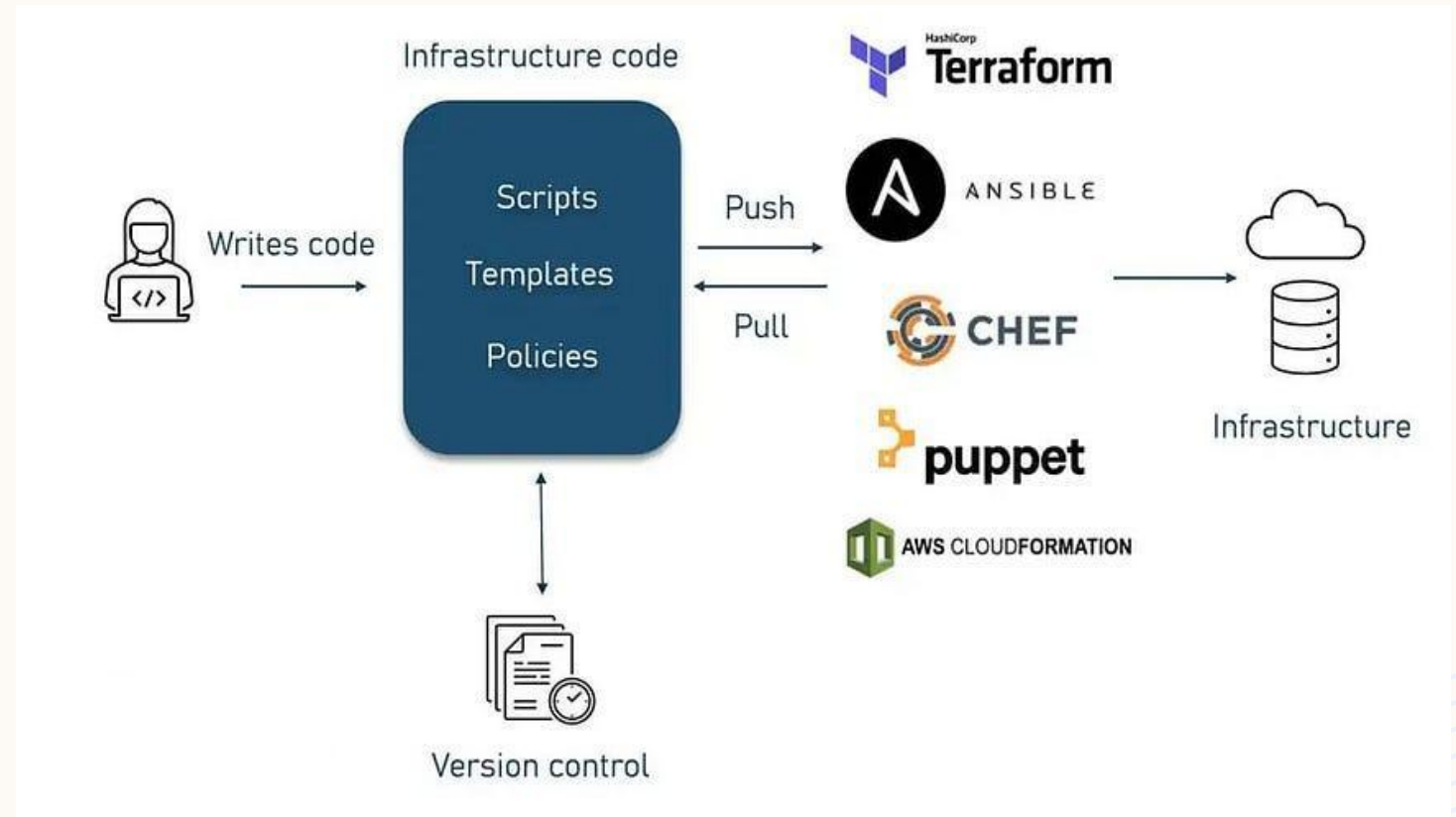
- In the event of failures, IaC makes it easier to rebuild or replicate infrastructure quickly, reducing downtime and enhancing fault tolerance.

How does Infrastructure as code work?

IaC uses software tools to automate administrative tasks by describing infrastructure in code.

It is implemented in the following ways:

1. The team writes the infrastructure configurations in a domain specific language.
2. The code files are pushed to a code repository.
3. An IaC tool runs the code and performs the required activities.



How IaC Works in DevOps

10

1. Writing Infrastructure Code

- Write Infrastructure as Code in domain specific language.
- Define infrastructure as code using YAML, HCL, JSON, or scripting languages.

2. Version Control

- Store infrastructure code in repositories (e.g., GitHub, GitLab, Bitbucket).
- Enables tracking changes, rollbacks, and collaboration.

3. Deployment

- Use tools like **Terraform Plan** or **Ansible Dry Run** to check for errors before deployment.
- Use CI/CD pipelines to deploy infrastructure automatically.
- Integrate with Jenkins, GitHub Actions, or GitLab CI/CD.
- Example: Terraform “apply” command to deploy resources.

Choosing an IaC tool

1. **APPROACH** - Declarative vs Imperative Approach.
2. **INFRASTRUCTURE TYPE** - Mutable vs. Immutable Infrastructure.
3. **FUNCTION** - Provisioning vs Configuration Management



Declarative Approach (What to do):

12

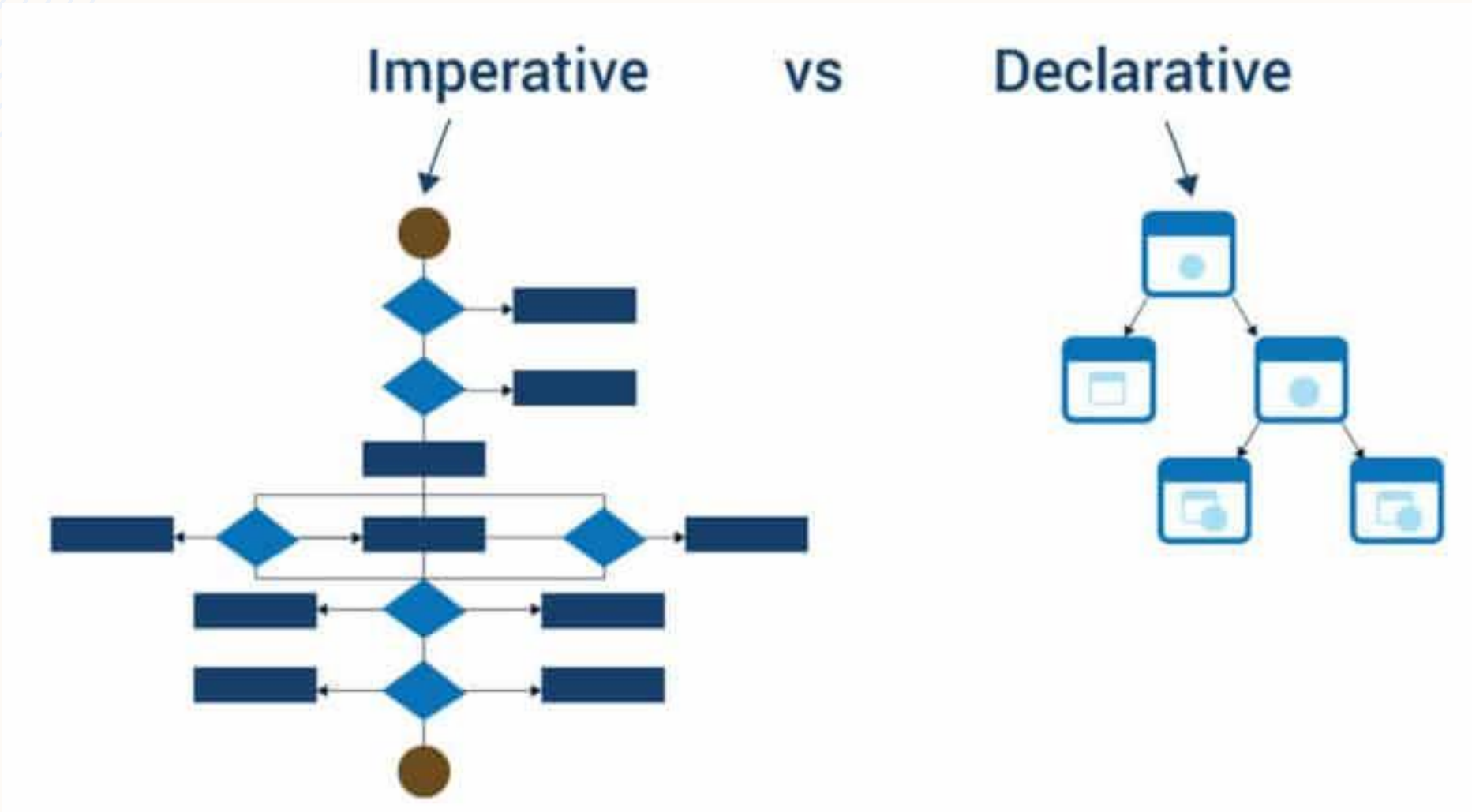
- You need to **define the desired final output of the infrastructure without going into details** on how it should be achieved, leaving the platform to handle it by itself.
- This method usually requires less knowledge on the user's part and it will always produce the same result.
- It is achieved with declarative languages such as SQL, YAML, or JSON.

Example: Terraform, CloudFormation, Ansible.

Imperative Approach (How to do it):

- You have to **define specific step-by-step commands** to achieve the required final output, it can be very detailed and create complex configurations, having more control over the task.
- But in this method of approach, a high level of skill is needed to write those commands with languages like Java, Ruby, or Python.

Example: Bash scripts, Kubernetes manifests.



Mutable infrastructure:

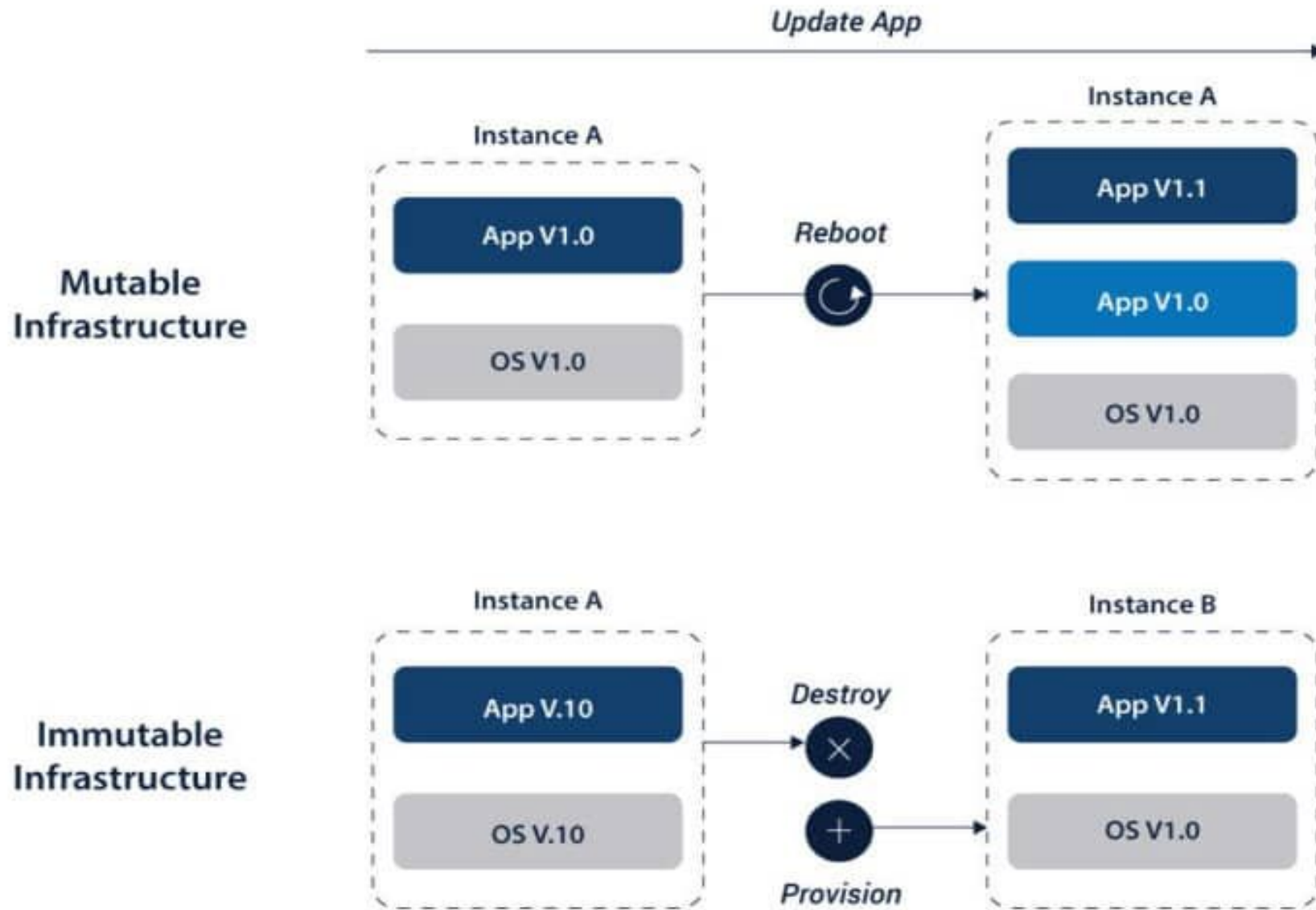
- It **can be changed** to fit business needs.
- You can easily introduce updates to the existing version, apply patches, and scale.

Example: Ansible.

Immutable infrastructure :

- It **can't be changed** once deployed.
- If you want to make alterations to it, you have to replace it with a new version.
- Since versions are independent, you can easily roll back, track mistakes, and enjoy consistency and predictability.

Example: Terraform, Docker containers.



Provisioning:

16

- It is the **process of setting up an IT infrastructure** (virtual machines, databases and so on) and making them available to the users.
- Provisioning is the process of **creating and setting up infrastructure resources** such as servers, databases, networks, and storage.

Example: Terraform, AWS CloudFormation

Configuration management:

- It comes after provisioning and it means **installing software, configuring servers desired state and maintaining them.**
- Configuration management ensures that provisioned infrastructure is **properly configured** with required settings, software, and dependencies.

Example: Ansible, Chef, Puppet

COMPARING INFRASTRUCTURE AS CODE TOOLS

Name	Languages	Function	Approach	Infrastructure type
 Terraform	HCL + Typescript, Python, Java, C#, Go with CDK	Provisioning	Declarative	Immutable
 AWS CLOUDFORMATION	JSON, YAML + TypeScript, Python, Java, .NET, and Go with CDK	Provisioning	Declarative	Both
 ANSIBLE	Python, Ruby, YAML	Configuration management	Imperative	Mutable
 puppet	PuppetDSL, YAML	Configuration management	Declarative	Mutable
 CHEF	Ruby	Configuration management	Declarative	Mutable

Tools for Implementing IaC

1. Terraform

- An open-source tool for declarative infrastructure provisioning.
- Uses HashiCorp Configuration Language (HCL).
- Supports multi-cloud (AWS, Azure, GCP) and on-premise environments.

2. AWS CloudFormation

- Amazon's declarative IaC tool for AWS cloud resources.
- Uses JSON or YAML to define resources.
- Integrates deeply with AWS services.

3. Ansible

- A configuration management tool used for automation.
- Uses YAML playbooks for defining configurations.
- Agentless, meaning it does not require installation on target machines.

4. Puppet

- Declarative configuration management tool.
- Uses its own DSL (Domain-Specific Language).
- Ideal for enforcing consistent configurations across large-scale environments.

5. Chef

- Imperative configuration management tool.
- Uses Ruby-based scripts called "cookbooks" and "recipes."
- Ensures consistent deployments across environments.



AWS CloudFormation



Puppet



Ansible



Terraform



Chef



Azure Resource
Manager

SALTSTACK

Saltstack



Vagrant



Pulumi

Pulumi

Example Use Cases of IaC

21

1. Provisioning Cloud Infrastructure

Automating AWS, Azure, or GCP infrastructure using Terraform or CloudFormation.

2. Setting Up Kubernetes Clusters

Deploying Kubernetes clusters using.

3. Configuring Virtual Machines

Using Ansible or Puppet to configure VM settings automatically.

4. Managing Hybrid Cloud Environments

Using Terraform to manage resources across multiple cloud providers.

5. Automating Network Infrastructure

Configuring firewalls, load balancers, and VPNs with IaC.

IMPLEMENTING IaC EXAMPLE - Provisioning and Configuring AWS EC2 Instance

Provisioning an AWS EC2 Instance using Terraform

22

```
resource "aws_instance" "web" {  
  ami      = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
}
```

This Terraform script **creates** an EC2 instance but does not configure what runs inside it.

Configuring Apache on an EC2 instance using Ansible

```
- hosts: web_servers  
  become: true  
  tasks:  
    - name: Install Apache  
      apt:  
        name: apache2  
        state: latest  
    - name: Start Apache Service  
      service:  
        name: apache2  
        state: started
```

This Ansible playbook **configures** an EC2 instance by installing Apache and starting the service.

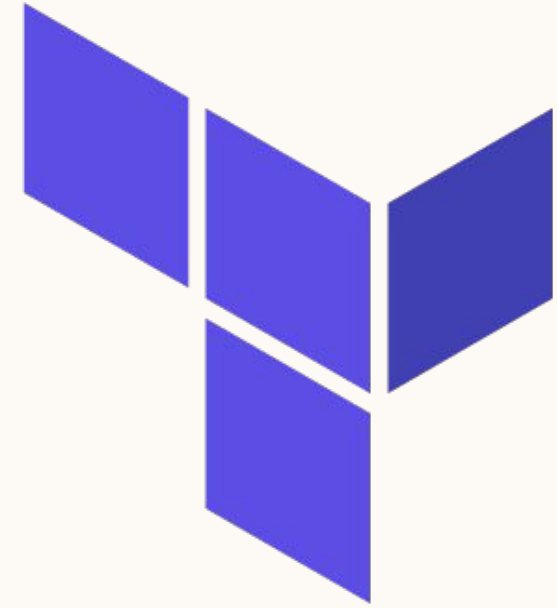
The background of the image features a large, light cream-colored circle centered on a dark blue field. This central circle is partially framed by light blue shapes on the left and light pink shapes on the right, creating a modern, abstract design.

TERRAFORM

What is Terraform?

24

- Terraform is an **open-source** tool that helps **automate the provisioning and deployment of infrastructure across various cloud providers** (AWS, Azure, GCP, etc.) **and on-premises environments.**
- Terraform is an open-source infrastructure provisioning tool created by HashiCorp
- It follows a **declarative approach**, meaning you describe the desired state of your infrastructure, and Terraform takes care of the provisioning.
- Terraform allows you to create, modify, and destroy infrastructure resources across different cloud providers, such as AWS, Azure, GCP, etc.

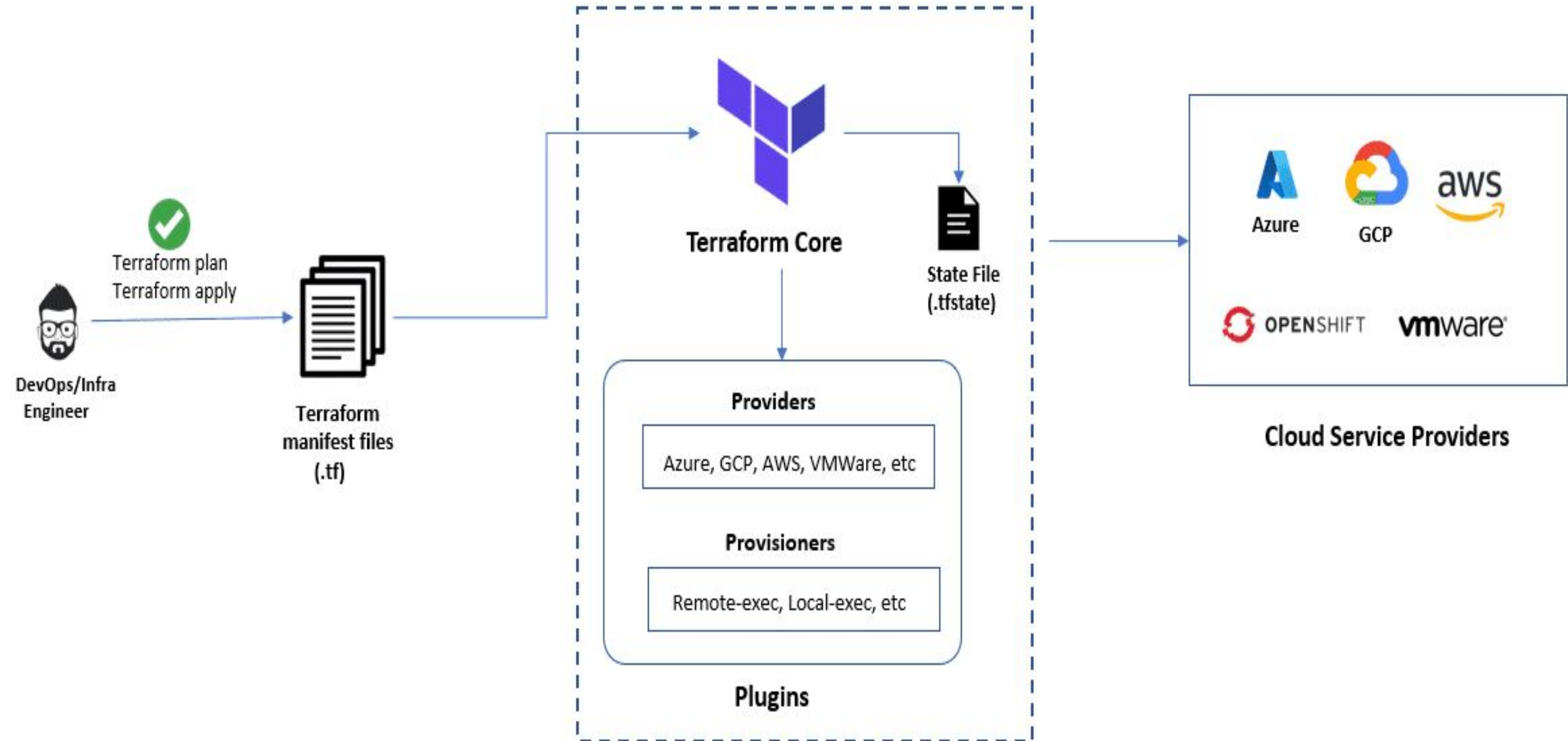


HashiCorp

Terraform

TERRAFORM ARCHITECTURE

25



1. DevOps/Infrastructure Engineer

- The engineer writes **Terraform files (.tf files)** to define infrastructure resources.
- Runs **Terraform commands** like “**terraform plan**” and “**terraform apply**” to provision infrastructure.

2. Terraform Manifest Files (.tf Files)

- These are the configuration files written in **HashiCorp Configuration Language (HCL)** or **JSON**.
- Define cloud resources like **VMs, databases, networking, and security policies**.

3. Terraform Core

27

- The Terraform Core is the **execution engine that reads .tf files, manages state, and interacts with providers.**
- Performs the following tasks:
 - ❑ **Reads** and validates configuration files.
 - ❑ **Plans** the execution of changes (terraform plan).
 - ❑ **Applies** changes to infrastructure (terraform apply).
 - ❑ **Stores** infrastructure state in the state file (terraform.tfstate).

4. State File (terraform.tfstate)

- Terraform maintains a state file **to track the current state** of managed infrastructure.
- Helps Terraform understand what changes are needed to match the desired state.
- Can be **stored locally or remotely** (S3, Terraform Cloud, etc.) for team collaboration.

5. Providers

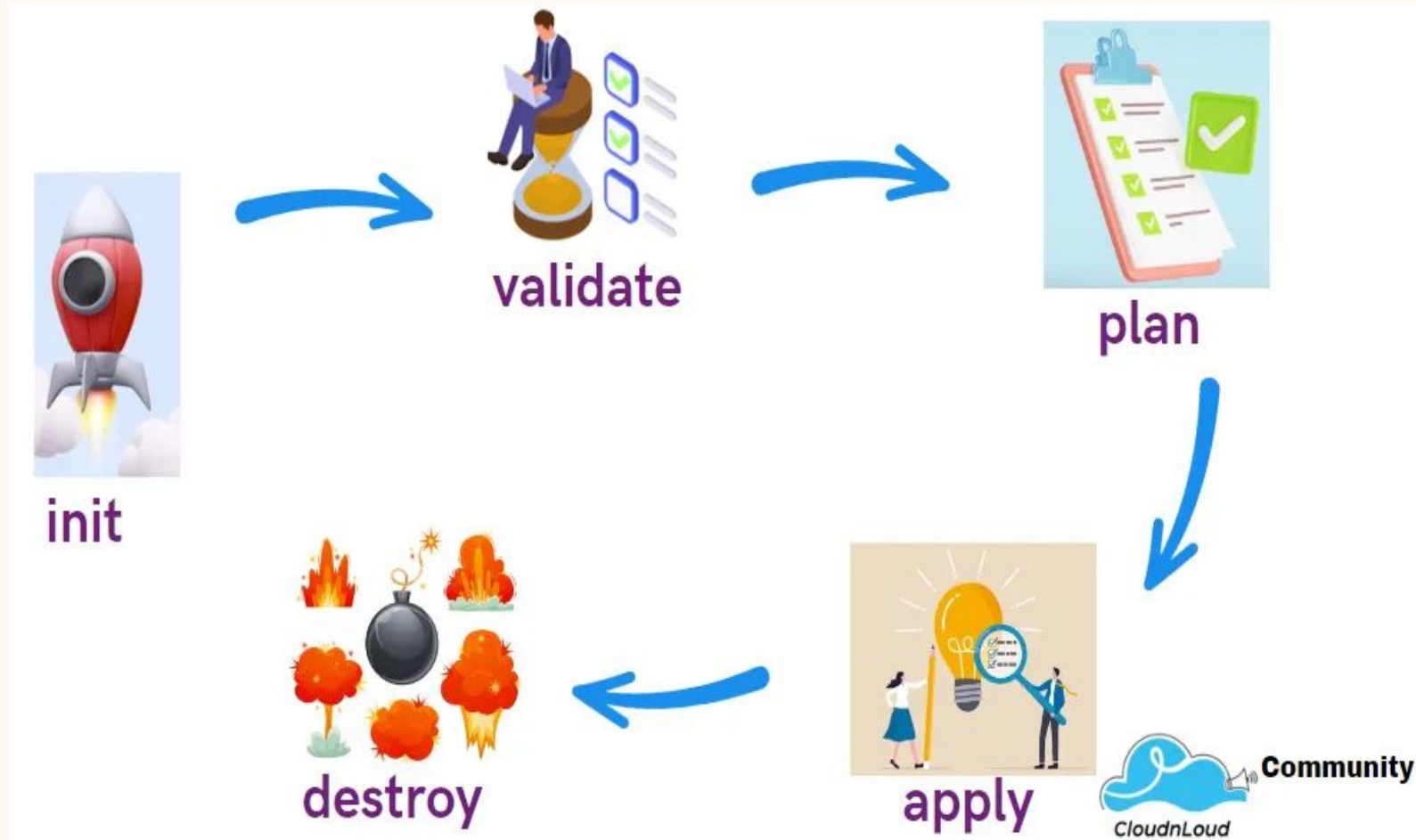
- Terraform interacts with cloud platforms using providers, such as - **AWS (terraform-provider-aws), Azure (terraform-provider-azure), Google Cloud (terraform-provider-google)**etc.
- Providers act as plugins that allow Terraform to manage resources in different environments.

6. Provisioners

- **Used to run scripts and provisioning commands** on resources **after they are created**.
- Common provisioners:
 - **remote-exec** – Runs commands on a remote machine.
 - **local-exec** – Runs commands on the local machine.

Terraform Workflow/Lifecycle

29



The workflow consists of lifecycle stages which start with the **init**, **validate**, **plan**, **apply**, and **destroy**.

The typical Terraform workflow consists of five main steps:

30

1. Initialize
2. Validate
3. Plan
4. Apply
5. Destroy

Workflow:

Write Configuration (.tf files)

- Define the desired infrastructure in HCL.

Initialize (terraform init)

- Terraform **initializes the working directory** containing terraform configuration files.
- Terraform initializes **downloads and installs the provider's plugin** so that it can later be executed.

Validate (terraform validate)

- The terraform validate command validates the configuration files in a directory.
- Validate command to **verify whether a configuration is syntactically valid**

Plan (terraform plan)

- Terraform plan command is used **to create an execution plan.**
- It **compares the desired terraform state with the current state** in the cloud. It doesn't change the deployment.

Apply (terraform apply)

- It **applies the changes required** to reach the desired state of the configuration.
- **Creates/modifies/deletes resources as per the plan.**

Destroy (terraform destroy)

- The Terraform destroy command is used to **delete the created resources** in the Terraform-managed infrastructure.

Terraform Configuration Syntax

```
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {  
# Block body  
<IDENTIFIER> = <EXPRESSION> # Argument  
}
```



Basic Terraform Configuration Example

33

```
provider "aws" {  
  region = "us-east-1"  
}
```

```
resource "aws_instance" "example" {  
  ami      = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"
```

```
  tags = {  
    Name = "TerraformInstance"  
  }  
}
```

What This Code Does in Simple Steps:

- Tells Terraform to use AWS as the cloud provider in the us-east-1 region.
- Defines an EC2 instance (virtual machine) with:
 - A specific OS image (AMI)
 - A small instance type (t2.micro)
 - A name tag for easy identification
- Terraform will create this instance when you run terraform apply

Benefits of Using Terraform

- 1. Infrastructure as Code** – Manage and provision infrastructure using code instead of manual processes.
- 2. Declarative Configuration** – Define the desired state of infrastructure, and Terraform ensures it matches.
- 3. Version Control** – Store Terraform configurations in Git or other versioning tools for collaboration.
- 4. Modules** – Reusable Terraform configurations that help maintain consistency and reduce duplication.
- 5. Provider Ecosystem** – Supports multiple cloud and on-prem providers like AWS, Azure, GCP etc.

Advantages of Using Terraform Modules

- **Code Reduction**

By using modules, you can significantly reduce the amount of code you need to write and maintain. Rather than repeating the same code, you can create a module and reference it multiple times, passing different parameters.

- **Consistency**

Modules help ensure consistency across your infrastructure. By standardizing configurations, you can mitigate the risk of configuration drift and ensure that all your environments are set up consistently.

- **Simplified Management**

Modules encapsulate complex components into a single unit, simplifying the process of managing your infrastructure. Even with complex, multi-layer architectures, modules can help to keep your configurations tidy and manageable.

Disadvantages of Terraform Modules

- **Complexity**

Modules can add complexity, especially when they are nested or highly parameterized. This complexity can make modules difficult to understand and maintain, particularly for larger teams and projects.

- **Versioning**

Versioning modules can be a challenge. When a module is updated, you need to carefully manage the versions used in your different environments to avoid unwanted changes.

- **Flexibility vs. Standardization**

While modules promote standardization, they may limit flexibility. A highly abstracted module may not cater to all use cases and could require changes to meet specific needs, which can undermine the principle of reusability.

The image features the word "ANSIBLE" in a bold, dark blue, sans-serif font, centered within a white circular area. This circle is set against a background composed of three distinct color regions: a light blue area on the left, a light pink area on the right, and a dark blue area at the bottom. The white circle is positioned such that it overlaps the top and sides of the dark blue area, creating a frame for the text.

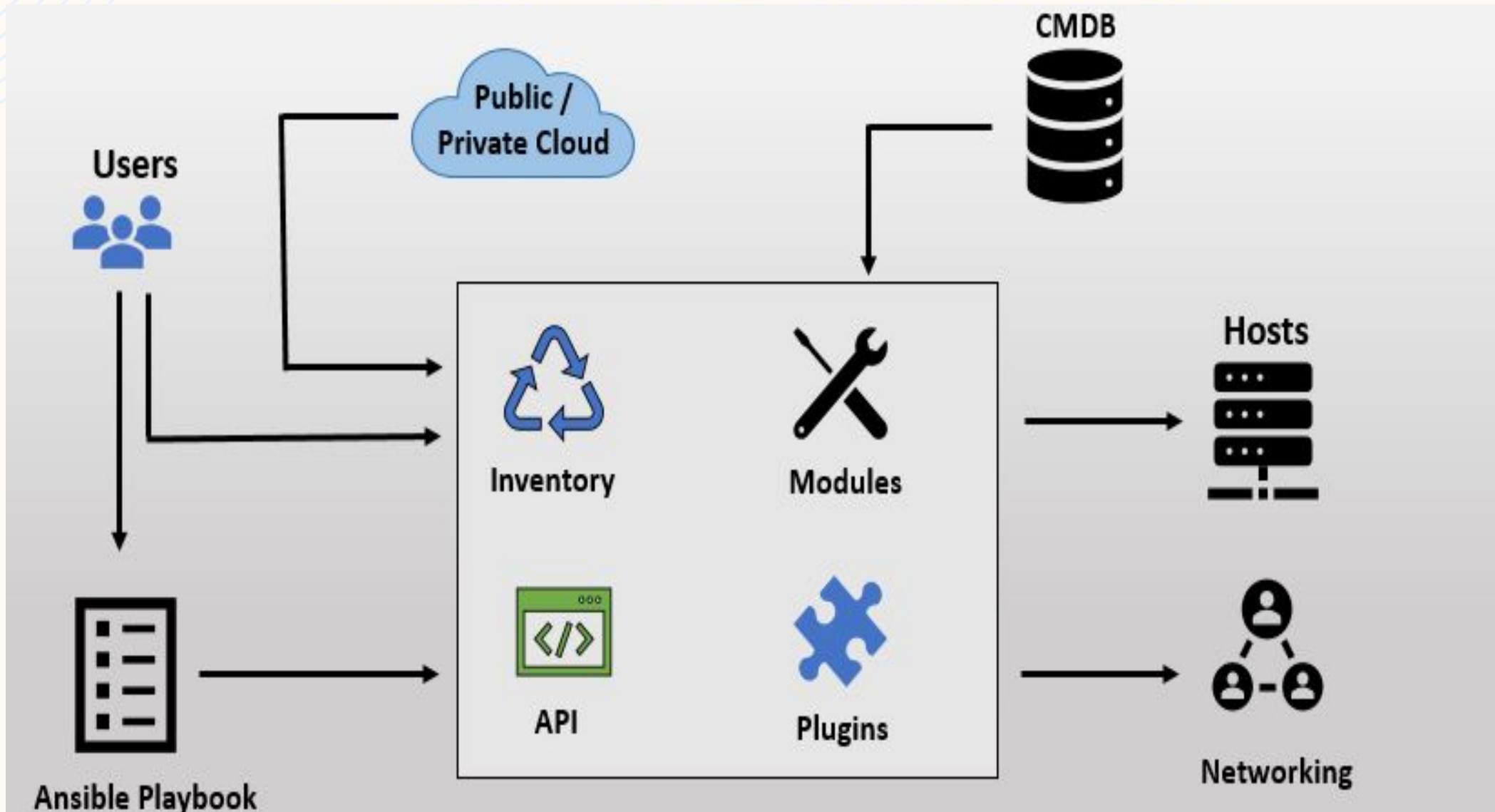
ANSIBLE

What is Ansible?

- Ansible is an **open-source** IT automation tool that simplifies **tasks like configuration management, application deployment, orchestration, and infrastructure as code (IaC)**.
- It is **agentless**, meaning it does not require software installation on target systems.
- It simplifies deploying, configuring, and managing systems using a language called **YAML(yet another markup language)**



ANSIBLE ARCHITECTURE



- 1. Inventories** – A list of servers with their IP addresses that Ansible manages using SSH (Linux) or WinRM (Windows).
- 2. APIs** – Connect Ansible with cloud services to automate tasks.
- 3. Modules** – Small programs that Ansible runs on remote servers to install software, manage files, or control services.
- 4. Plugins** – Extra features that enhance Ansible's functionality, such as logging, caching, and running tasks efficiently.

5. Networking – Ansible can also manage network devices (routers, switches, firewalls) without needing an agent.

6. Hosts – Computers or servers that Ansible controls and automates.

7. Playbooks – YAML files that define a set of tasks to configure or manage systems.

8. CMDB (Configuration Management Database) – A database that stores IT asset details and their relationships.

9. Cloud – A network of remote servers used for storing and managing data instead of using local machines.

Overall Workflow of Ansible.

- 1.** User writes playbook in YAML
- 2.** Playbook is sent to Ansible engine
- 3.** Engine checks inventory for servers
- 4.** Modules execute tasks
- 5.** Commands run on hosts or network devices

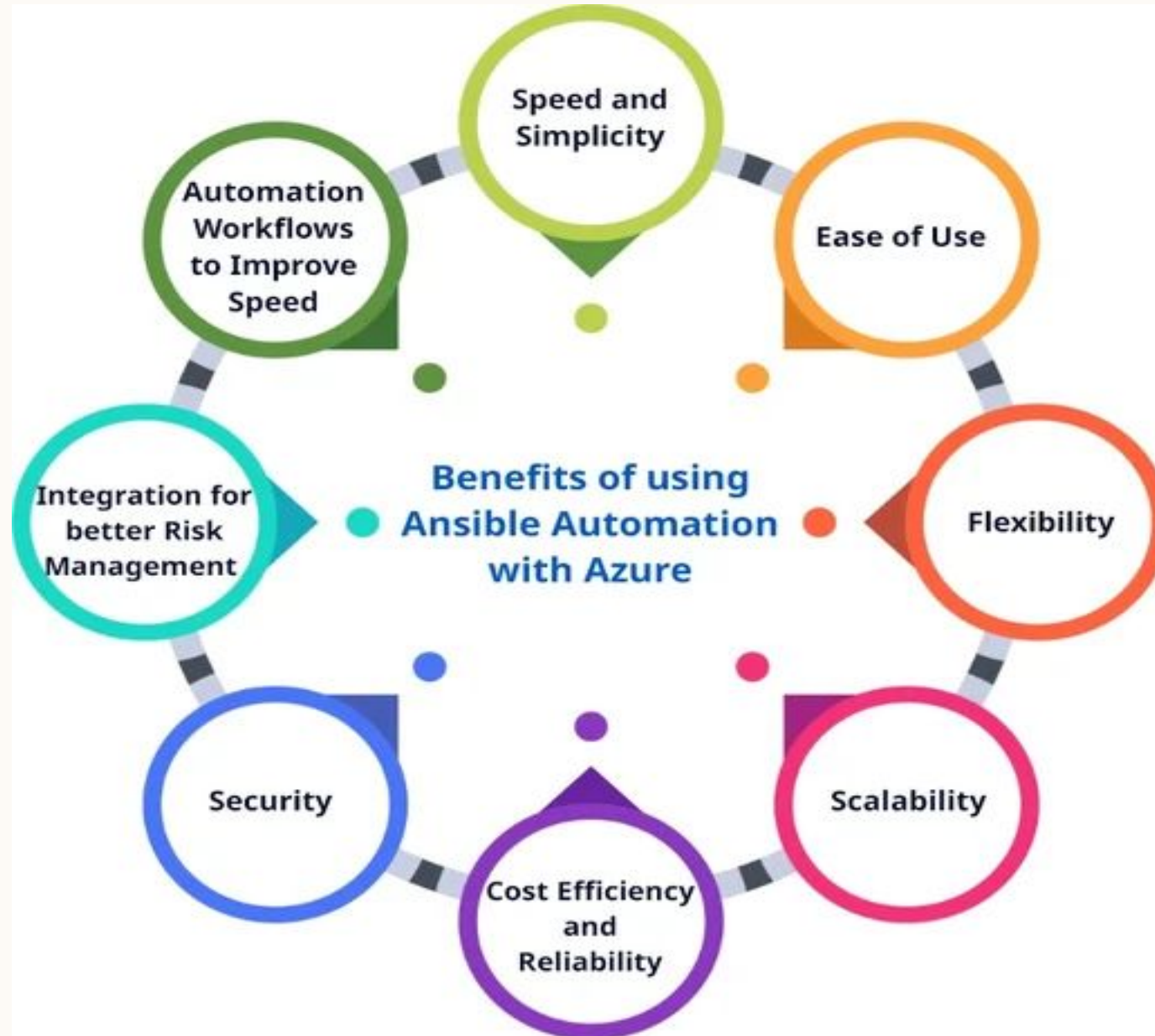
Example Configuration in Ansible

43

- name: Install and configure Apache web server
hosts: webservers
become: true
tasks:
 - name: Install Apache package
 - name: httpd
 - state: present
 - name: Start and enable Apache service
 - name: httpd
 - state: started
 - enabled: true
 - name: Create a basic HTML file
 - content: "<html><body><h1>Hello from Ansible!</h1></body></html>"
 - dest: /var/www/html/index.html
 - owner: apache
 - group: apache
 - mode: '0644'

Benefits Of Ansible

44



Benefits of Ansible

Speed and Simplicity – Automates complex workflows efficiently with minimal setup.

Ease of Use – Uses simple, human-readable YAML playbooks for automation.

Flexibility – Adapts to various environments, from on-premises to cloud.

Scalability – Manages and automates infrastructure at any scale.

Cost Efficiency and Reliability – Reduces manual intervention, lowering operational costs.

Security – Ensures compliance and secure automation with role-based access control.

Integration for Better Risk Management – Seamlessly integrates with Cloud provider services to enhance security and governance.

Automation Workflows to Improve Speed – Streamlines IT processes, reducing deployment time.

Advantages of Ansible

46

Agentless – No need to install agents on managed nodes, reducing complexity.

Simple & Easy to Learn – Uses human-readable **YAML playbooks**, making it beginner-friendly

Cross-Platform Support – Works with **Linux, Windows, cloud services, and network devices**.

Fast & Efficient – Uses **SSH** for direct execution, avoiding unnecessary overhead.

Extensible with Modules & Plugins – Over **450+ built-in modules** for automation.

Security & Compliance – Supports **Ansible Vault** for encrypting sensitive data like passwords.

Supports Cloud Automation – Easily integrates with **AWS, Azure, Google Cloud, and Kubernetes**.

Disadvantages of Ansible

Limited Windows Support – Primarily designed for Linux; Windows automation requires additional setup.

YAML Formatting Sensitivity – YAML is strict with indentation, leading to errors if misformatted.

No Centralized Logging – Unlike tools like **Puppet**, Ansible does not have built-in logging without external solutions.

Real-Time Monitoring is Limited – Lacks a real-time dashboard unless using **Ansible Tower (AWX)**.

Weak Version Control – Changes in playbooks are not **automatically tracked**, unlike in Terraform.

Difference between Terraform and Ansible

47

Feature	Terraform	Ansible
Main Purpose	Infrastructure provisioning	Configuration management
What it Creates	Cloud infrastructure (servers, networks, storage)	Software setup and server configuration
Language Used	HCL (HashiCorp Configuration Language)	YAML
Execution Style	Declarative	Mostly procedural
Typical Use	Create AWS EC2, VPC, Load balancers	Install software like Apache, Nginx, Docker
Cloud Support	Strong cloud infrastructure management	Used after infrastructure is created